УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У НОВОМ САДУ

Бане Герић

ЈСД за опис пословних процеса

ЗАВРШНИ РАД

Мастер академске студије

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ ● **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА** 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Бане Герић	Број индекса:	R232/2023
Степен и врста студија:	Мастер академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Игор Дејановић		

НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА:

- проблем – тема рада;
- начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;

НАСЛОВ ЗАВРШНОГ РАДА:

JСД за опис пословних процеса

ТЕКСТ ЗАДАТКА:

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Руководилац студијског програма:	Ментор рада:

Примерак за: □ - Студента; □ - Ментора

УНИВЕРЗИТЕТ У НОВОМ САДУ ● **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА**
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, **РБР**:		
Идентификациони број, **ИБР**:		
Тип документације, **ТД**:		Монографска документација
Тип записа, **ТЗ**:		Текстуални штампани материјал
Врста рада, **ВР**:		Дипломски - мастер рад
Аутор, **АУ**:		Бане Герић
Ментор, **МН**:		Др Игор Дејановић, редовни професор
Наслов рада, **НР**:		ЈСД за опис пословних процеса
Језик публикације, **ЈП**:		српски/ћирилица
Језик извода, **ЈИ**:		српски/енглески
Земља публиковања, **ЗП**:		Република Србија
Уже географско подручје, **УГП**:		Војводина
Година, **ГО**:		2026
Издавач, **ИЗ**:		Ауторски репринт
Место и адреса, **МА**:		Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, **ФО**: (поглавља/страна/ цитата/табела/слика/графика/прилога)		7/53/13/1/11/0/2
Научна област, **НО**:		Електротехничко и рачунарско инжењерство
Научна дисциплина, **НД**:		Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, **ПО**:		Шаблон, завршни рад, упутство
УДК		
Чува се, **ЧУ**:		У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, **ВН**:		
Извод, **ИЗ**:		Овај документ представља упутство за писање завршних радова на Факултету техничких наука Универзитета у Новом Саду. У исто време је и шаблон за Typst.
Датум прихватања теме, **ДП**:		
Датум одбране, **ДО**:		01.01.2025
Чланови комисије, **КО**:	Председник:	Др Петар Петровић, ванредни професор
	Члан:	Др Марко Марковић, доцент
	Члан:	Потпис ментора
	Члан, ментор:	Др Игор Дејановић, редовни професор

Образац **Q2.НА.04-05** - Издање 1

UNIVERSITY OF NOVI SAD ● **FACULTY OF TECHNICAL SCIENCES**
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, **ANO**:	
Identification number, **INO**:	
Document type, **DT**:	Monographic publication
Type of record, **TR**:	Textual printed material
Contents code, **CC**:	
Author, **AU**:	Bane Gerić
Mentor, **MN**:	Igor Dejanović, Phd., full professor
Title, **TI**:	DSL for business processes
Language of text, **LT**:	Serbian
Language of abstract, **LA**:	Serbian/English
Country of publication, **CP**:	Republic of Serbia
Locality of publication, **LP**:	Vojvodina
Publication year, **PY**:	2026
Publisher, **PB**:	Author's reprint
Publication place, **PP**:	Novi Sad, Dositeja Obradovica sq. 6
Physical description, **PD**: (chapters/pages/ref./tables/pictures/graphs/appendixes)	7/53/13/1/11/0/2
Scientific field, **SF**:	Electrical and Computer Engineering
Scientific discipline, **SD**:	Applied computer science and informatics
Subject/Key words, **S/KW**:	Template, thesis, tutorial
UC	
Holding data, **HD**:	The Library of Faculty of Technical Sciences, Novi Sad
Note, **N**:	
Abstract, **AB**:	This document provides guidelines for writing final theses at the Faculty of Technical Sciences, University of Novi Sad. At the same time, it serves as a Typst template.
Accepted by the Scientific Board on, **ASB**:	
Defended on, **DE**:	01.01.2025
Defended Board, **DB**: President:	Petar Petrović, Phd., assoc. professor
Member:	Marko Marković, Phd., asist. professor
Member:	
Member, Mentor:	Igor Dejanović, Phd., full professor

Menthor's sign

Obrazac **Q2.HA.04-05** - Izdanje 1

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

_____

Кандидат

_____

Садржај

1 Увод . . . 1
1.1 Мотивација и проблематика . . . 1
1.2 Циљ рада . . . 2
1.3 Методологија . . . 2
1.4 Структура рада . . . 3
2 Теоријски оквир и стање у области . . . 5
2.1 Језици специфични за домен . . . 5
2.2 Мета-моделовање и моделом вођен развој . . . 5
2.3 Управљање пословним процесима . . . 6
2.4 Поређење приступа . . . 6
2.5 Формални модели workflow система . . . 7
2.6 Статичка генерација наспрам динамичке интерпретације . . . 7
2.7 Поређење са постојећим алатима . . . 8
2.8 Генерација кода . . . 8
2.9 Позиционирање BPML приступа . . . 9
3 Дизајн и архитектура BPML језика . . . 11
3.1 Архитектура система . . . 11
3.2 Структура BPML модела . . . 12
3.3 Формални (EBNF) приказ граматике и кључне ријечи језика . . . 15
3.4 Семантичка валидација . . . 18
3.5 Генератори кода . . . 19
3.6 Архитектурни дијаграм система . . . 20
3.7 Структурни дијаграм имплементације . . . 20
3.8 Дијаграм процесног модела . . . 21
3.9 Дијаграм зависности задатака . . . 21
3.10 Дијаграм односа метамодела . . . 21
3.11 Улога процесне машине . . . 22
3.12 Ограничења тренутне верзије . . . 22
4 Генерисање backend система . . . 23
4.1 Архитектура генерисаног система . . . 23
4.2 Сервисни слој . . . 24
4.3 Конфигурација и покретање . . . 26
4.4 Предности аутоматске генерације backend-а . . . 27
5 Генерисање frontend система . . . 29
5.1 Архитектура генерисане React апликације . . . 29

5.2 Визуелизација процесног графа . . . 31
5.3 Навигација и рутирање . . . 32
5.4 Предности аутоматске генерације frontend-а . . . 32
6 Валидација и анализа система . . . 33
6.1 Синтаксна валидација . . . 33
6.2 Семантичка валидација . . . 33
6.3 Функционална валидација backend система . . . 34
6.4 Функционална валидација frontend система . . . 34
6.5 Студија случаја: Систем одобравања фактура . . . 34
6.6 Перформансе и скалабилност . . . 35
6.7 Предности предложеног приступа . . . 35
6.8 Ограничења система . . . 35
6.9 Општа оцена система . . . 35
7 Закључак . . . 37
7.1 Будући рад . . . 38
Списак слика . . . 39
Списак листинга . . . 41
Списак табела . . . 43
Списак коришћених скраћеница . . . 45
Списак коришћених појмова . . . 47
Биографија . . . 49
Литература . . . 51
Грешке у литературним наводима . . . 53

Глава 1

Увод

Савремени информациони системи све чешће укључују комплексне пословне процесе који обухватају више актера, стања, правила одлучивања и међусобно зависних активности. Имплементација таквих система класичним програмским приступом доводи до расипања логике процеса кроз више слојева апликације, што отежава одржавање, еволуцију система и формалну анализу понашања.

Насупрот томе, језици специфични за домен (ЈСД, енг. Domain-Specific Languages - DSL) омогућавају моделовање проблема на вишем нивоу апстракције, употребом концепата блиских самом домену примјене. Према Дејановићу, језици специфични за домен представљају рачунарске језике који повећавају експресивност ограничавајући се на уско дефинисану област проблема и њене концепте [1]. Овај приступ омогућава већу читљивост, бољу контролу исправности и могућност аутоматске обраде модела.

У области управљања пословним процесима (енг. Business Process Management – BPM) често се користе стандардизоване нотације као што је BPMN, али оне углавном захтијевају додатну ручну имплементацију или интеграцију са постојећим системима. Истраживања показују да алати засновани на моделима и генерацији кода могу значајно убрзати развој и смањити број грешака [2]. Управо тај приступ представља основу овог рада.

Централна идеја рада јесте пројектовање и имплементација екстерног језика специфичног за домен, названог BPML (Business Process Modeling Language), који омогућава декларативно дефинисање пословних процеса и аутоматско генерисање комплетне full-stack апликације. Језик омогућава дефинисање ентитета, улога, стања, задатака и транзиција, уз формалну семантичку валидацију модела. На основу једног модела генерише се:

- backend систем заснован на Spring Boot платформи,
- frontend апликација реализована употребом React и TypeScript технологија,
- процесна машина за управљање извршавањем процеса,
- механизам контроле приступа заснован на улогама.

1.1 Мотивација и проблематика

У класичном приступу развоју пословних апликација логика процеса често је имплицитно уграђена у сервисни слој, контролере или чак кориснички интерфејс. Овакав приступ отежава формалну провјеру исправности, увођење измјена и поновну упо-

требу логике у другим системима. Додатно, не постоји јединствена спецификација која би недвосмислено описивала ток процеса.

Мотивација за развој BPML језика произилази из потребе за:

- јасном и формалном спецификацијом пословних процеса,
- централизацијом процесне логике,
- аутоматском провјером семантичке исправности,
- смањењем количине ручно писаног инфраструктурног кода,
- убрзањем развоја прототипа и продукционих система.

1.2 Циљ рада

Основни циљ рада је пројектовање, имплементација и анализа језика специфичног за домен намијењеног моделовању пословних процеса, као и развој генератора кода који из датог модела производи функционалну full-stack апликацију.

Специфични циљеви рада су:

- дефинисање формалне граматике језика употребом TextX алата,
- имплементација семантичких провјера модела,
- дизајн архитектуре генератора кода,
- генерисање backend система заснованог на Spring Boot технологији,
- генерисање frontend система заснованог на React и TypeScript технологијама,
- имплементација процесне машине за управљање стањима и задацима,
- анализа предности и ограничења предложеног приступа.

1.3 Методологија

У раду је примијењен приступ заснован на моделом вођеном развоју (енг. Model-Driven Development – MDD). Граматика језика дефинисана је употребом TextX метајезика, који омогућава генерисање метамодела и апстрактног синтаксног стабла на основу текстуалне спецификације [2].

Након синтаксне анализе, над моделом се извршавају семантичке провјере које обезбјеђују:

- јединственост назива елемената,
- исправност референци између ентитета,
- валидност транзиција између стања,
- исправну употребу улога и задатака.

Генерисање кода реализовано је употребом Jinja шаблонског механизма, чиме се омогућава трансформација модела у изворни код циљних платформи. На тај начин се једном дефинисана спецификација процеса преводи у потпуно функционалан софтверски систем.

1.4 Структура рада

Рад је организован у седам поглавља.

У другом поглављу дат је теоријски оквир који обухвата концепте језика специфичних за домен и моделовања пословних процеса.

Треће поглавље описује дизајн BPML језика, његову синтаксу, семантику и архитектуру пројекта.

Четврто и пето поглавље анализирају механизам генерисања backend и frontend система.

У шестом поглављу представљена је валидација система на конкретним примјерима, као и анализа предности и ограничења приступа.

Седмо поглавље садржи закључак и правце будућег развоја.

Глава 2

Теоријски оквир и стање у области

Развој сложених информационих система захтијева јасну апстракцију доменских концепата, формализацију пословне логике и могућност аутоматске трансформације спецификације у извршиву имплементацију. У том контексту, језици специфични за домен, моделом вођен развој и системи за управљање пословним процесима представљају кључне теоријске основе овог рада.

2.1 Језици специфични за домен

Језици специфични за домен представљају рачунарске језике чија је експресивност усмјерена на уско дефинисану проблемску област. За разлику од језика опште намјене, који настоје да буду универзални, DSL-ови ограничавају синтаксу и семантику на појмове релевантне одређеном домену.

Фаулер дефинише DSL као језик намијењен изражавању решења унутар специфичног проблемског простора, при чему се повећава читљивост и смањује семантички јаз између домена и имплементације [3]. Мерник и сарадници истичу да DSL-ови омогућавају повећање продуктивности, смањење грешака и бољу комуникацију између доменских експерата и програмера [4].

Према класификацији, DSL-ови могу бити:

- интерни (уграђени у постојећи језик),
- екстерни (са сопственом синтаксом и граматиком).

У овом раду обрађен је екстерни DSL, што омогућава потпуну контролу над синтаксом и независност од ограничења језика опште намјене. Екстерни приступ је погодан када је потребно формално дефинисати структуру модела и обезбиједити строгу семантичку валидацију.

2.2 Мета-моделовање и моделом вођен развој

Моделом вођен развој (MDD) представља парадигму у којој модели имају централну улогу у процесу изградње система. Шмит истиче да MDD омогућава подизање нивоа апстракције и аутоматизацију трансформације модела у имплементацију [5].

Основни концепти укључују:

- метамодел – дефиницију структуре модела,
- модел – конкретну инстанцу метамодела,

- трансформацију – процес превођења модела у други модел или изворни код.

OMG иницијатива Model-Driven Architecture (MDA) формализује овај приступ кроз раздвајање платформски независног модела (PIM) и платформски специфичног модела (PSM). У контексту овог рада, BPML модел представља платформски независну спецификацију, док се Spring Boot и React код могу посматрати као платформски специфичне имплементације.

TextX алат омогућава дефинисање граматике и аутоматско генерисање метамодела на основу текстуалне спецификације [2]. Овај приступ омогућава:

- формалну синтаксну анализу,
- изградњу апстрактног синтаксног стабла,
- имплементацију семантичких правила.

2.3 Управљање пословним процесима

Пословни процес дефинише се као скуп међусобно повезаних активности које трансформишу улазне ресурсе у излазне вриједности. Думас и сарадници описују процес као структурирану секвенцу активности са јасно дефинисаним стањима и правилима прелаза [6].

Основни елементи процесног модела су:

- стања,
- активности,
- транзиције,
- актери,
- пословни објекти.

У пракси, BPMN представља најшире прихваћену нотацију за моделовање процеса. BPMN омогућава визуелну спецификацију, али се често ослања на засебне engine механизме за извршавање процеса.

Веске истиче да workflow engine мора обезбиједити контролу стања, синхронизацију паралелних активности и управљање улогама [7]. Многи постојећи системи заснивају се на интерпретацији процесног модела током извршавања.

2.4 Поређење приступа

Постоје два доминантна приступа имплементацији пословних процеса:

1. Интерпретативни приступ Процесни модел се чува у бази података и интерпретира током извршавања (нпр. BPM engine системи).

2. Генеративни приступ Модел се трансформише у изворни код који се компајлира и извршава као дио апликације.

Интерпретативни приступ омогућава динамичке измјене процеса, али уводи додатни runtime слој и повећава сложеност система. Генеративни приступ, који је примијењен у овом раду, обезбјеђује:

- боље перформансе,
- лакше дебаговање,
- јаснију структуру кода,
- директну везу између спецификације и имплементације.

2.5 Формални модели workflow система

У области управљања пословним процесима значајан допринос дали су формални модели засновани на Петријевим мрежама. Ван дер Алст показује да Петријеве мреже представљају математички формализам погодан за моделовање стања, паралелизма и синхронизације у workflow системима [8].

Петријеве мреже омогућавају:

- формалну верификацију процеса,
- анализу deadlock ситуација,
- провјеру достижности стања,
- анализу паралелизма.

У каснијим радовима, Ван дер Алст проширује анализу кроз концепт process mining-а, који омогућава анализу извршених процеса на основу логова из система [9].

У контексту BPML језика, концепти стања, транзиција и зависности задатака могу се посматрати као поједностављена апстракција формалних workflow модела. Иако BPML не користи директно Петријеве мреже, његова структура је у складу са основним принципима формалног моделовања процеса.

2.6 Статичка генерација наспрам динамичке интерпретације

У пракси се могу уочити два приступа извршавању процеса:

1. Динамичка интерпретација
2. Статичка генерација

Код динамичке интерпретације, процесни модел се учитава и тумачи током извршавања. Овај приступ користе класични BPM engine системи. Предност овог приступа је могућност измјене процеса без поновне компилације система. Међутим, овај модел уводи додатни runtime слој који:

- повећава сложеност система,
- отежава дебаговање,
- уводи додатну латенцију.

Са друге стране, статичка генерација подразумијева трансформацију модела у изворни код који се компајлира као дио апликације. Овај приступ је у складу са принципима моделом вођеног развоја [5].

Предности статичке генерације су:

- боље перформансе (без интерпретационог слоја),
- јаснија структура кода,
- лакше тестирање,
- директна повезаност спецификације и имплементације.

Недостатак овог приступа је потреба за поновном генерацијом и компилацијом у случају измјене процеса.

BPML приступ свјесно се ослања на статичку генерацију, што га позиционира као генеративни DSL систем, а не као класични интерпретативни workflow engine.

2.7 Поређење са постојећим алатима

Савремени системи као што су Camunda или Activiti заснивају се на BPMN спецификацији и интерпретативном engine моделу. Они омогућавају динамичку измјену процеса, али захтијевају посебну конфигурацију и runtime инфраструктуру.

Супротно томе, BPML интегрише процесни модел директно у апликациони код, чиме се:

- поједностављује архитектура,
- смањује зависност од спољашњих engine система,
- обезбјеђује ближа интеграција са доменским моделом.

Ова разлика представља кључну теоријску и практичну одлуку у дизајну система.

2.8 Генерација кода

Аутоматска генерација кода представља кључни механизам моделом вођеног развоја. Кливленд наглашава да генератори омогућавају елиминацију понављајућих шаблонских дијелова и повећање конзистентности система [10].

У овом раду генерација се заснива на шаблонском механизму који користи:

- структуру модела,
- мапирање типова кореспондентних језика,
- контекст трансформације.

Овај приступ омогућава да једна спецификација резултира комплетном backend и frontend апликацијом.

2.9 Позиционирање BPML приступа

На основу анализе постојећих решења може се закључити да:

- BPMN системи доминирају у визуелном моделовању,
- workflow engine системи омогућавају динамичко извршавање,
- MDD алати омогућавају трансформацију модела,
- DSL алати омогућавају формалну спецификацију домена.

BPML интегрише ове концепте у јединствен систем који:

- користи екстерни DSL за спецификацију процеса,
- примјењује статичку семантичку валидацију,
- генерише комплетну full-stack имплементацију,
- укључује процесну машину и визуелизацију.

Овакво позиционирање омогућава да се BPML посматра као практична синтеза DSL и MDD концепата у домену управљања пословним процесима.

Глава 3

Дизајн и архитектура BPML језика

BPML (Business Process Modeling Language) представља екстерни језик специфичан за домен намијењен моделовању пословних процеса и аутоматској генерацији full-stack софтверских система. Језик је имплементиран употребом TextX алата, док је трансформација модела у изворни код реализована путем Jinja шаблонског механизма.

Дизајн језика заснива се на принципима:

- јасне декларативне синтаксе,
- формалног метамодела,
- статичке семантичке валидације,
- генерације кода засноване на моделу.

3.1 Архитектура система

Систем је организован у три логичка слоја:

1. Слој дефиниције језика
2. Слој семантичке обраде
3. Слој генерације кода

Слој дефиниције језика обухвата TextX граматику (`bpml.tx`), којом се дефинише синтакса језика. На основу граматике TextX генерише метамодел и апстрактно синтаксно стабло.

Сљедећи исјечак приказује дио TextX граматике којом се дефинишу процеси, задаци и транзиције:

```
Process:
    'process' name=ID '{'
        elements*=ProcessElement
    '}'
;

Task:
    'task' name=ID
    'in' state=[State]
    ('by' role=[Role] | 'auto')
    ('depends_on' dependencies+=[Task][','])?
;

Transition:
    'transition' name=ID
    'from' from_state=[State]
    'to' to_state=[State]
    ('by' role=[Role])?
;
```

Листинг 1: Исјечак TextX граматике за BPML језик

Ова граматика омогућава:

- јасно раздвајање синтаксних конструката,
- референтно повезивање елемената (нпр. state=[State]),
- дефинисање зависности између задатака.

TextX на основу ове спецификације аутоматски гради метамодел и апстрактно синтаксно стабло.

Слој семантичке обраде реализован је кроз модул за валидацију модела (`processors.py`). У овом слоју се врше провјере конзистентности и исправности модела пре фазе генерације кода.

Слој генерације кода реализован је кроз два генератора:

- Spring Boot backend генератор
- React frontend генератор

Оба генератора користе Jinja шаблоне и заједнички контекст изграђен на основу модела.

3.2 Структура BPML модела

BPML модел састоји се из:

- блока пројекта,
- једног или више процеса.

Процес представља централну јединицу спецификације и садржи:

- ентитете,
- улоге,
- стања,
- задатке,
- транзиције.

Ова структура омогућава потпуно описивање животног циклуса пословног процеса.

3.2.1 Ентитети и типски систем

Ентитети представљају доменске објекте који се чувају у бази података и над којима се извршавају операције током процеса. Сваки ентитет садржи атрибуте са јасно дефинисаним типовима.

Подржани су основни типови:

- string
- int
- float
- boolean

Поред основних типова, подржани су и набројиви типови (enum). Типски систем је проширен мапирањем на циљне платформе. У слоју уграђених типова дефинисане су мапе за:

- Java типове (нпр. string → String, int → Integer),
- TypeScript типове (нпр. string → string, int → number).

Ова апстракција омогућава независност доменског модела од имплементационих детаља циљних технологија.

3.2.2 Улоге и хијерархија

Улоге дефинишу актере процеса и омогућавају моделовање хијерархије надређености. Хијерархија се користи за контролу приступа током извршавања задатака и транзиција.

Семантичка правила обезбјеђују:

- јединственост назива улога,
- забрану самонадгледања,
- исправност наведених референци.

Овај механизам представља основу за role-based access control у генерисаном систему.

3.2.3 Стања и транзиције

Стања представљају фазе животног циклуса процеса. Транзиције дефинишу контролисане прелазе између стања и повезане су са одређеним улогама. Имплицитни механизам паралелизма у BPML језику заснива се на декларативном опису више грана унутар једног стања, без потребе за експлицитним навођењем join конструкције. Свака грана представља независан ток извршавања који може садржати један или више задатака. Семантика извршавања дефинисана је тако да се све гране покрећу паралелно, док се завршетак стања сматра постигнутим тек након што су сви задаци из свих грана успјешно извршени.

Овакво рјешење поједностављује синтаксу језика и смањује вербалну редундантност, а притом задржава пуну изражајност у моделовању паралелних токова. Имплементација на нивоу генерације кода осигурава коректно праћење статуса задатака и синхронизацију грана у процесној машини.

Семантичке провјере обезбјеђују:

- да изворно и одредишно стање постоје,
- да стања нису идентична,
- да наведена улога постоји у процесу.

Овај модел директно се пресликава у процесну машину генерисаног backend система.

3.2.4 Задаци и зависности

Задатак (Task) представља јединицу рада унутар одређеног стања. Задаци могу бити:

- ручни (додељени улози),
- аутоматски.

Систем намеће правило да задатак мора бити или ручни или аутоматски, али не и оба истовремено.

Подржане су зависности између задатака. Уколико више задатака нема међусобних зависности, могу се извршавати паралелно. Уколико задатак зависи од више претходних задатака, представља join тачку у процесном току.

Семантички слој обезбеђује:

- да зависности референцирају постојеће задатке,
- да задатак не зависи од самог себе,
- да не постоје недефинисане референце.

Додатно, семантичка анализа провјерава да зависности између задатака не формирају циклус. Другим ријечима, није дозвољено да задатак директно или индиректно зависи од самог себе. Ова провјера елиминише могућност недостижних стања у извршавању и спречава блокирање процеса усљед кружних предуслова.

3.2.5 Дизајнерске одлуке

Током развоја BPML језика разматране су различите синтаксне и семантичке варијанте за моделовање паралелних токова. Један од разматраних приступа био је увођење експлицитне синтаксе за fork и branch конструкције, на примјер:

```
state OnboardingState {
    fork {
        branch ITSetup: SetupWorkstation, AssignEquipment
        branch Training: ScheduleOrientation, ConductTraining
    }
}
```

Овакав приступ чини паралелизам изричитим и ближим формалним workflow моделима. Међутим, анализа употребљивости показала је да таква синтакса уноси редундансу, повећава комплексност језика и оптерећује корисника додатним декларацијама.

Усвојено је једноставније рјешење у којем је паралелизам имплицитан. Задаци који немају међусобне зависности могу се извршавати паралелно, док се синхронизација остварује путем зависности (join тачака). Овим приступом задржана је минималистичка синтакса, а семантика паралелизма је и даље јасно дефинисана.

Ова одлука је донесена са циљем очувања једноставности језика и смањења когнитивног оптерећења приликом моделовања процеса.

3.3 Формални (EBNF) приказ граматике и кључне ријечи језика

```
(* Лексички симболи (апстракција): *)
ID      = /* идентификатор */ ;
STRING  = /* ниска под наводницима */ ;

(* Почетни симбол: *)
BusinessProcessModel
  = ProjectInfo , { Process } ;

ProjectInfo
  = "project" , "{" ,
      "name" , ":" , ID ,
      [ "description" , ":" , STRING ] ,
      [ "version" , ":" , STRING ] ,
      [ "author" , ":" , STRING ] ,
    "}" ;

Process
  = "process" , ID , "{" ,
      Entity ,
      Role ,
```

```
      State ,
      Task ,
      Transition ,
    "}" ;

Entity
  = "entity" , ID , "{" ,
      { Attribute } ,
    "}" ;

Attribute
  = ID , ":" , DataType ;

DataType
  = "int" | "float" | "string" | "boolean" | Enum ;

Enum
  = "enum" , "{" , ID , { "," , ID } , "}" ;

Role
  = "role" , ID , [ "supervises" , ID , { "," , ID } ] ;

State
  = "state" , ID ;

Transition
  = "transition" , ID ,
    "from" , ID ,
    "to" , ID ,
    "by" , ID ;

Task
  = "task" , ID ,
    "in" , ID ,
    ( ("by" , ID) | "auto" ) ,
    [ "affects" , ID , { "," , ID } ] ,
    [ "depends_on" , ID , { "," , ID } ] ;

Comment
  = "//" , { ? било који знак осим краја реда ? } ;
```

Листинг 2: Формални EBNF приказ синтаксе BPML језика

Овим EBNF приказом формално је описана комплетна синтакса BPML језика, укључујући све кључне ријечи и правила појављивања елемената. Референце на елементе (нпр. `state=[State]`, `role=[Role]`) у изворној TextX граматици у EBNF приказу поје-

ностављено су представљене као `ID`, док се њихова исправност додатно осигурава семантичком валидацијом.

У табели X дат је систематски преглед свих кључних ријечи BPML језика и њихове семантичке улоге.

Табела 1: Систематизација кључних ријечи BPML језика

Кључна ријеч	**Категорија**	**Опис семантике**
project	Метаподаци	Дефинише основне информације о пројекту.
name	Метаподаци	Назив пројекта или елемента.
description	Метаподаци	Текстуални опис пројекта.
version	Метаподаци	Верзија модела или система.
author	Метаподаци	Аутор модела.
process	Структура	Дефинише нови пословни процес.
entity	Доменски модел	Дефинише пословни ентитет са атрибутима.
enum	Типски систем	Дефинише набројив тип.
int	Типски систем	Цјелобројни тип података.
float	Типски систем	Децимални број.
string	Типски систем	Текстуални тип.
boolean	Типски систем	Логичка вриједност (true/false).
role	Контрола приступа	Дефинише учесника у процесу.
supervises	Контрола приступа	Дефинише хијерархију улога.
state	Процесни модел	Представља стање у животном циклусу процеса.
initial	Процесни модел	Означава почетно стање.
final	Процесни модел	Означава завршно стање.
task	Извршавање	Дефинише оперативни корак унутар стања.
in	Извршавање	Повезује задатак са стањем.
by	Извршавање	Означава улогу која извршава задатак или транзицију.

Кључна ријеч	**Категорија**	**Опис семантике**
auto	Извршавање	Аутоматски задатак без корисничке интервенције.
depends_on	Извршавање	Дефинише зависност између задатака.
affects	Извршавање	Означава ентитете на које задатак утиче.
transition	Процесни модел	Дефинише прелаз из једног стања у друго.
from	Процесни модел	Полазно стање транзиције.
to	Процесни модел	Циљно стање транзиције.
fork	Паралелизам	Означава почетак паралелног гранања.
branch	Паралелизам	Дефинише појединачну грану унутар fork конструкције.

Оваква систематизација кључних ријечи показује да BPML језик комбинује декларативно моделовање процеса, доменских ентитета и контроле приступа унутар јединствене формалне синтаксе.

3.4 Семантичка валидација

Након синтаксне анализе, над моделом се извршава статичка семантичка провјера. Ова фаза спречава генерисање неконзистентног или непотпуног кода.

Провјере обухватају:

- дупликате процеса, ентитета, улога, стања, задатака и транзиција,
- валидност свих референци,
- исправност структуре процеса,
- исправност хијерархије улога,
- исправност зависности задатака.

Грешке се пријављују у фази парсирања, чиме се обезбјеђује рано откривање проблема. Поред провјере да ли све референце у хијерархији улога показују на постојеће улоге, систем спроводи и провјеру конзистентности хијерархије како би се спријечиле кружне зависности (циклуси). На тај начин није могуће дефинисати улогу која је (директно или индиректно) надређена сама себи, чиме се осигурава да је структура надређености увијек ациклична.

Семантичка провјера модела имплементирана је у Python слоју обраде. Сљедећи исјечак илуструје провјеру дупликата и референци:

```python
def validate_unique_names(elements, element_type):
    names = set()
    for e in elements:
        if e.name in names:
            raise Exception(f"Duplicate {element_type}: {e.name}")
        names.add(e.name)

def validate_task_dependencies(task):
    for dep in task.dependencies:
        if dep == task:
            raise Exception(f"Task {task.name} cannot depend on itself")
```

Листинг 3: Исјечак семантичке валидације у Python-у

Оваквим приступом грешке се откривају у фази парсирања, прије генерације кода.

Систем такође спречава кружне зависности у хијерархији улога и зависностима задатака. Пошто се све референце провјеравају током обраде модела, није могуће дефинисати:

- улогу која је директно или индиректно надређена сама себи,
- задатак који директно или индиректно зависи од самог себе.

Овим се обезбјеђује ацикличност хијерархија и стабилност процесног тока.

3.5 Генератори кода

Генерација кода заснива се на принципу трансформације модела у изворни код. На основу BPML модела гради се контекст који садржи:

- назив пројекта,
- списак процеса,
- списак ентитета,
- стања,
- задатке,
- транзиције.

Spring Boot генератор производи:

- JPA ентитете,
- репозиторијуме,
- сервисни слој,
- REST контролере,
- процесну машину,
- конфигурационе фајлове.

React генератор производи:

- CRUD компоненте за ентитете,
- сервисни слој за комуникацију са backend-ом,
- компоненте за управљање процесима,
- визуелизацију процесног графа.

Оба генератора користе заједничке филтере за мапирање типова и форматирање идентификатора.

3.6 Архитектурни дијаграм система

Архитектура BPML система може се посматрати као цјелина која обухвата фазу моделовања, фазу семантичке обраде и фазу генерације кода. На слици 1 приказан је логички архитектурни приказ система.

[BPML модел (.bpml)] ↓ [TextX парсер и метамодел] ↓ [Семантичка валидација] ↓ [Контекст генератора] ↙ ↘ [Spring Boot генератор] [React генератор] ↓ ↓ [Backend апликација] [Frontend апликација]

Слика 1: Слика 1: Логичка архитектура BPML система

Архитектура се састоји од сљедећих корака:

1. Дефинисање модела у BPML синтакси.
2. Парсирање модела употребом TextX алата.
3. Формирање метамодела и апстрактног синтаксног стабла.
4. Извршавање семантичких провјера.
5. Генерисање контекста за шаблонски механизам.
6. Генерисање backend и frontend кода.

Оваква архитектура омогућава јасно раздвајање доменске спецификације од инфраструктурне имплементације.

3.7 Структурни дијаграм имплементације

Структура Python пројекта организована је у више модула, како је приказано на слици 2.

bpml/ ├── language/ │ ├── bpml.tx │ ├── processors.py │ ├── builtins.py │ └── custom_model.py │ ├── generator/ │ ├── springboot/ │ ├── react/ │ └── util/ │ └── examples/

Слика 2: Слика 2: Структура BPML имплементације

Модул `language` садржи граматику, семантичке процесоре и дефиницију типског система. Модул `generator` садржи имплементацију генератора и Jinja шаблоне. Модул `util` садржи помоћне функције за мапирање типова, форматирање идентификатора и рад са фајл системом.

Ова организација омогућава:

- изолацију синтаксе од генерације,
- једноставно проширење новим циљним платформама,
- независно тестирање семантичког слоја.

3.8 Дијаграм процесног модела

BPML модел описује процес као скуп стања, задатака и транзиција. На слици 3 приказан је поједностављени пример процесног модела одобравања фактуре.

[Draft] │ ▼ [Submitted] │ ┌──────────────────────┴──────────────────────┐ ▼ ▼ [Approved] [Rejected] │
▼ [Completed]

Слика 3: Слика 3: Пример процесног модела одобравања фактуре

Унутар стања могу постојати више задатака. Уколико задаци немају међусобне зависности, извршавају се паралелно (fork семантика). Уколико задатак зависи од више претходних, представља join тачку.

3.9 Дијаграм зависности задатака

На слици 4 приказан је пример fork-join структуре задатака.

StartTask │ ┌──────┴──────┐ ▼ ▼ TaskA TaskB └──────┬──────┘ ▼ FinalTask

Слика 4: Слика 4: Fork-join семантика задатака

У овом примјеру:

- `TaskA` и `TaskB` се могу извршавати паралелно,
- `FinalTask` почиње тек након завршетка оба претходна задатка.

Ова структура директно се пресликава у процесну машину backend система.

3.10 Дијаграм односа метамодела

Метамодел BPML језика може се представити као UML дијаграм класа, приказан на слици 5.

Project └── Process[*]

Process ├── Entity[**]** ├── **Role[**] ├── State[**]** ├── **Task[**] └── Transition[*]

Task ├── state → State ├── role → Role (опционо) └── dependencies → Task[*]

Transition ├── from_state → State ├── to_state → State └── role → Role

Слика 5: Слика 5: Поједностављени UML метамодел BPML језика

Метамодел дефинише односе између кључних елемената и представља основу за:

- семантичку анализу,
- провјеру конзистентности,
- изградњу контекста за генерацију.

3.11 Улога процесне машине

Генерисани backend систем садржи процесну машину која управља:

- тренутним стањем инстанце процеса,
- активним задацима,
- дозвољеним транзицијама,
- статусом процеса.

Процесна машина користи податке из метамодела како би обезбиједила да су све промјене стања у складу са дефинисаном спецификацијом.

Оваквим приступом обезбјеђује се директна кореспонденција између модела и извршне логике, чиме се минимизује ризик од одступања између спецификације и имплементације.

3.12 Ограничења тренутне верзије

Иако BPML даје потпуну спецификацију линеарних и разгранатих процеса, тренутна верзија има одређена ограничења:

- не постоји подршка за условне изразе у транзицијама,
- не постоји динамичко креирање улога,
- не постоји графички едитор (само статички приказ у виду графа стања и задатака),
- процесна логика је статички дефинисана.

Увођење условних израза омогућило би да се одлуке у процесу формално изразе на нивоу модела, као што су праг износа, статус валидације или вриједности атрибута ентитета. Као будући рад разматра се проширење синтаксе транзиција условима и дефинисање семантике њихове евалуације у процесној машини.

Ова ограничења представљају основу за даљи развој језика.

Глава 4

Генерисање backend система

На основу спецификације дефинисане у BPML моделу, систем омогућава аутоматску генерацију backend апликације засноване на Spring Boot платформи. Генерисана апликација представља потпуно функционалан серверски систем који обухвата:

- доменски модел,
- слој перзистенције,
- сервисни слој,
- REST интерфејс,
- процесну машину,
- механизме контроле приступа.

Циљ овог приступа је да се доменска логика дефинише декларативно у DSL-у, док се инфраструктурни код генерише аутоматски.

4.1 Архитектура генерисаног система

Генерисана backend апликација прати слојевиту архитектуру. Основни слојеви су:

1. Модел (Entity слој)
2. Репозиторијуми
3. Сервисни слој
4. Контролерски слој
5. Процесна машина

Сваки ентитет дефинисан у BPML моделу резултира генерисањем:

- JPA ентитета,
- Spring Data репозиторијума,
- сервисне класе,
- REST контролера,
- DTO и мапера.

Овим приступом обезбјеђује се конзистентна структура кода за све доменске објекте.

4.1.1 Генерисање JPA ентитета

За сваки BPML ентитет генерише се Java класа означена JPA анотацијама. Атрибути ентитета мапирају се на одговарајуће Java типове у складу са типским мапирањем дефинисаним у слоју уграђених типова.

Подржано је:

- генерисање примарног кључа,
- мапирање enum типова,
- повезивање ентитета са процесном инстанцом.

Сваки ентитет садржи поље које омогућава повезивање са инстанцом процеса, чиме се омогућава праћење података у контексту конкретног процеса.

4.1.2 Репозиторијуми и перзистенција

За сваки ентитет генерише се Spring Data репозиторијум који омогућава:

- основне CRUD операције,
- аутоматску имплементацију упита,
- интеграцију са JPA механизмом.

Подржане су базе података као што су H2 (развојно окружење) и PostgreSQL (продукција).

Перзистенција процесних инстанци и задатака реализована је преко засебних ентитета који омогућавају чување:

- тренутног стања процеса,
- статуса процеса,
- временских ознака,
- додељених улога,
- JSON варијабли процеса.

4.2 Сервисни слој

Сервисни слој представља апстракцију између контролера и слоја перзистенције. За сваки ентитет генерише се сервис који:

- обрађује пословну логику,
- позива репозиторијуме,
- управља трансакцијама,
- врши валидацију захтјева.

Поред ентитетских сервиса, генеришу се и специјализовани сервиси за:

- управљање процесним инстанцама,
- управљање задацима,
- извршавање транзиција.

4.2.1 Процесна машина

Кључни елемент backend система представља процесна машина. Она осигурава:

- креирање нових инстанци процеса,
- праћење тренутног стања,
- активирање задатака у одговарајућем стању,
- валидацију транзиција,
- евиденцију извршавања.

Стања, задаци и транзиције дефинисани у BPML моделу претварају се у статичку конфигурацију процесне машине. На тај начин се гарантује да runtime понашање директно одговара спецификацији модела.

Статуси процеса могу бити:

- RUNNING,
- COMPLETED,
- SUSPENDED,
- TERMINATED,
- ERROR.

Задаци имају сопствени животни циклус (PENDING, затим IN_PROGRESS и на крају COMPLETED или CANCELED).

Транзиције између стања врше се искључиво ако:

- корисник посједује одговарајућу улогу,
- процесна инстанца се налази у очекиваном стању,
- нису прекршена правила дефинисана моделом.

Ова правила имплементирана су у сервисном слоју, чиме се спречава неовлашћена или нелогична промјена стања.

Генерисана процесна машина заснива се на статички дефинисаној структури стања и транзиција. Сљедећи исјечак приказује дио Jinja шаблона:

```
  public void executeTransition(String transitionName, ProcessInstance
instance) {
      if (!isValidTransition(transitionName, instance.getCurrentState())) {
          throw new IllegalStateException("Invalid transition");
      }

    instance.setCurrentState(getTargetState(transitionName));
    instanceRepository.save(instance);
}
```

Листинг 4: Исјечак генерисаног Java кода за извршавање транзиција

Овај механизам осигурава да су све транзиције у складу са моделом дефинисаним у BPML спецификацији.

Сљедећи исјечак приказује дио генерисаног сервиса за управљање задацима:

```
public void completeTask(Long taskId) {
    TaskInstance task = taskRepository.findById(taskId)
        .orElseThrow(() -> new RuntimeException("Task not found"));

    task.setStatus(TaskStatus.COMPLETED);
    taskRepository.save(task);
}
```

Листинг 5: Исјечак генерисаног Java кода за завршетак задатка

Животни циклус задатака директно је дефинисан моделом и имплементиран у сервисном слоју.

4.2.2 REST интерфејс

За сваки ентитет и процес генеришу се REST контролери који омогућавају:

- креирање и измјену ентитета,
- покретање процеса,
- преузимање активних задатака,
- извршавање транзиција,
- праћење статуса инстанци.

REST API омогућава интеграцију са frontend системом и другим спољашњим сервисима.

4.2.3 Обрада изузетака

Систем генерише централизовани механизам за обраду изузетака. На тај начин обезбјеђује се:

- конзистентан формат одговора,
- јасна порука о грешци,
- раздвајање техничких и пословних изузетака.

4.3 Конфигурација и покретање

Генерисани пројекат садржи:

- Maven конфигурацију,
- application.properties фајл,
- CORS конфигурацију,
- иницијализацију процесних дефиниција.

Покретањем апликације учитава се дефиниција процеса, након чега је систем спреман за креирање инстанци и управљање процесима.

4.4 Предности аутоматске генерације backend-а

Аутоматска генерација backend система пружа:

- конзистентну архитектуру,
- уштеду времена при развоју,
- смањење инфраструктурног кода,
- директну везу између модела и имплементације,
- једноставније одржавање система.

Овакав приступ омогућава да се фокус развоја помјери са техничке имплементације на моделовање домена.

Глава 5

Генерисање frontend система

На основу BPML модела и генерисаног backend система, BPML генератор омогућава аутоматску изградњу frontend апликације засноване на React оквиру и TypeScript језику. Генерисана апликација представља комплетан кориснички интерфејс за управљање ентитетима и процесним инстанцама.

Циљ frontend генерације је:

- обезбјеђивање конзистентног корисничког интерфејса,
- елиминација ручног писања понављајућег CRUD кода,
- омогућавање визуелног праћења процеса,
- директна интеграција са REST API интерфејсом backend система.

5.1 Архитектура генерисане React апликације

Генерисана апликација организована је у више логичких цјелина:

1. Типске дефиниције (TypeScript интерфејси)
2. CRUD странице за ентитете
3. Сервисни слој (API комуникација)
4. Компоненте за управљање процесима
5. Заједничке (shared) компоненте

Оваква структура омогућава раздвајање одговорности и лако проширење система.

5.1.1 Типске дефиниције и ентитети

На основу BPML спецификације генеришу се TypeScript интерфејси који одговарају backend ентитетима. Типови атрибута се мапирају из доменских типова у TypeScript типове (нпр. string → string, int → number).

За enum типове генеришу се TypeScript набројиве вредности, чиме је омогућена типска сигурност у корисничком интерфејсу.

Овај приступ омогућава:

- статичку провјеру типова,
- смањење runtime грешака,
- синхронизацију модела између frontend-а и backend-а.

5.1.2 CRUD интерфејси за ентитете

За сваки ентитет генеришу се сљедеће компоненте:

- Листа (List) – приказ свих инстанци ентитета,
- Дијалог (Dialog) – форма за креирање и измену,
- Детаљна страница (Page) – приказ појединачне инстанце.

Компоненте користе Material-UI библиотеку како би се пружио конзистентан и модеран изглед.

CRUD функционалност омогућава:

- креирање нових записа,
- измену постојећих,
- брисање,
- претрагу и филтрирање.

Сљедећи исјечак приказује дио генерисане React листе ентитета:

```
const {{ entity.name }}List = () => {
  const [data, setData] = useState<{{ entity.name }}[]>([]);

  useEffect(() => {
    api.getAll().then(setData);
  }, []);

  return (
    <Table>
      {data.map(item => (
        <TableRow key={item.id}>
          <TableCell>{item.id}</TableCell>
        </TableRow>
      ))}
    </Table>
  );
};
```

Листинг 6: Исјечак генерисане React компоненте за листу ентитета

Компонента користи React hook механизам и сервисни слој за комуникацију са backend-ом.

5.1.3 Сервисни слој и комуникација са backend-ом

За сваки ентитет генерише се сервис који комуницира са REST API интерфејсом backend-а. Сервисни слој обухвата:

- HTTP захтеве (GET, POST, PUT, DELETE),
- обраду одговора,

- централизовано руковање грешкама.

Овај слој омогућава изолацију логике комуникације од UI компоненти.

5.1.4 Управљање процесним инстанцама и задацима

Frontend систем омогућава:

- покретање нових процесних инстанци,
- приказ активних и завршених процеса,
- увид у тренутно стање,
- преглед историје транзиција.

За сваки процес генеришу се:

- Dashboard страница,
- Листа инстанци,
- Детаљна страница инстанце,
- Листа активних задатака.

Ова структура омогућава кориснику потпун увид у животни циклус процеса.

С друге стране, задаци представљају централни елемент интеракције корисника са процесом. Генерисане компоненте омогућавају:

- приказ задатака по улогама,
- преузимање задатка,
- извршавање задатка,
- приказ статуса.

Систем обезбеђује да су доступни само задаци који су валидни у тренутном стању процеса.

5.2 Визуелизација процесног графа

Једна од напредних функционалности генерисаног frontend-а јесте визуелизација процесног модела. Визуелизација процеса заснива се на граф репрезентацији у којој су стања чворови вишег нивоа, док су транзиције усмјерене гране између њих. Оваква репрезентација омогућава интуитивно праћење тока процеса и јасно истицање гранања и конвергенције токова. Поред тога, унутар стања се приказују задаци као чворови нижег нивоа, што омогућава да се истовремено уоче и контролни ток (стања/транзиције) и ток активности (задаци/зависности).

На основу спецификације, генерише се интерактивни граф који приказује:

- стања као контејнере,
- задатке унутар стања,
- транзиције између стања,

- зависности између задатака.

На сљедећој слици приказан је примјер визуелизације процесног графа:

Граф се генерише статички у фази генерације кода и користи библиотеке за аутоматско позиционирање чворова.

Ова визуелизација омогућава:

- лакше разумевање структуре процеса,
- идентификацију паралелних токова,
- јасно приказивање fork-join семантике.

5.3 Навигација и рутирање

Генерисана апликација користи систем рутирања који омогућава:

- прелазак између ентитетских страница,
- приступ процесним dashboard-има,
- директно адресирање појединачних инстанци.

Руте се генеришу аутоматски на основу назива процеса и ентитета.

5.4 Предности аутоматске генерације frontend-а

Аутоматска генерација frontend система доноси сљедеће предности:

- значајно смањење ручног рада,
- конзистентност UI компоненти,
- типску сигурност,
- синхронизацију са backend моделом,
- брзу изградњу прототипа.

Овим приступом омогућено је да се једном дефинисани BPML модел користи као централни извор истине (енг. Source of truth) за цјелокупан систем.

Глава 6

Валидација и анализа система

У овом поглављу анализира се исправност, функционалност и ограничења предложеног BPML приступа. Валидација система обухвата:

- синтаксну исправност модела,
- семантичку конзистентност,
- исправност генерисаног backend система,
- исправност генерисаног frontend система,
- анализу предности и ограничења.

6.1 Синтаксна валидација

Синтаксна валидација врши се употребом TextX парсера на основу дефинисане граматике. У овој фази проверава се:

- исправност структуре фајла,
- валидност кључних речи,
- исправност блокова (project, process, entity, role, state, task, transition),
- коректност типских дефиниција.

Уколико модел не задовољава синтаксна правила, процес генерације се прекида и пријављује се одговарајућа грешка.

6.2 Семантичка валидација

Након успешног парсирања, извршава се семантичка анализа модела. Циљ ове фазе је откривање логичких и структурних грешака које није могуће уочити само синтаксном провером.

Семантичка валидација обухвата:

- проверу јединствености назива процеса,
- проверу јединствености ентитета унутар процеса,
- проверу јединствености улога,
- проверу јединствености стања,
- проверу јединствености задатака,
- проверу јединствености транзиција,
- проверу исправности референци (state, role, entity),
- проверу исправности зависности задатака,
- проверу да задатак није истовремено аутоматски и ручни,

- проверу да транзиција не повезује идентична стања.

Овај механизам омогућава откривање грешака у раној фази, пре генерисања кода.

6.3 Функционална валидација backend система

Функционална исправност backend система проверавана је кроз сљедеће сценарије:

1. Креирање новог ентитета.
2. Покретање процесне инстанце.
3. Активирање задатака у иницијалном стању.
4. Извршавање транзиције уз проверу улоге.
5. Прелазак у сљедеће стање.
6. Завршетак процеса.

Тестирањем је потврђено да:

- процесна машина не дозвољава нелегалне транзиције,
- корисници без одговарајуће улоге не могу извршити транзицију,
- статуси процеса и задатака се исправно ажурирају,
- подаци ентитета остају повезани са одговарајућом инстанцом процеса.

6.4 Функционална валидација frontend система

Frontend систем тестирањем је потврдио:

- исправан приказ ентитета,
- валидност CRUD операција,
- исправан приказ активних задатака,
- правилну синхронизацију са backend API-јем,
- исправан приказ процесног графа.

Типска сигурност TypeScript-а додатно доприноси раном откривању грешака у комуникацији са backend-ом.

6.5 Студија случаја: Систем одобравања фактура

За потребе анализе коришћен је пример процеса одобравања фактура који садржи:

- више улога (нпр. запослени, менаџер, финансијски сектор),
- више стања,
- паралелне задатке,
- гранање одлука (одобравање или одбијање).

Тестирање је показало да:

- паралелни задаци функционишу у складу са fork семантиком,
- join механизам захтева завршетак свих зависних задатака,

- стања се мењају искључиво у складу са дефинисаним транзицијама,
- визуелизација процеса одговара дефинисаном моделу.

6.6 Перформансе и скалабилност

С обзиром на то да се процесна логика генерише статички, runtime извршавање не захтева динамичко тумачење DSL-а. Ово доприноси:

- бољим перформансама,
- једноставнијем дебаговању,
- мањој комплексности runtime окружења.

Скалабилност система зависи од:

- конфигурације базе података,
- хоризонталног скалирања Spring Boot апликације,
- конфигурације инфраструктуре.

6.7 Предности предложеног приступа

Анализом система уочене су сљедеће предности:

- централизација процесне логике у DSL-у,
- елиминација дуплирања инфраструктурног кода,
- конзистентност између модела и имплементације,
- брза изградња прототипа,
- једноставно проширење новим процесима,
- типска сигурност у оба слоја система.

6.8 Ограничења система

Упркос наведеним предностима, уочена су и одређена ограничења:

- недостатак подршке за условне изразе у транзицијама,
- ограничена подршка за сложене типове података,
- одсуство динамичке конфигурације процеса,
- непостојање графичког едитора модела,
- одсуство напредних аналитичких механизама.

Ова ограничења представљају основу за будући развој и проширење BPML језика.

6.9 Општа оцена система

На основу спроведене анализе може се закључити да предложени BPML приступ успешно интегрише:

- DSL моделовање,
- семантичку валидацију,

- генерацију кода,
- процесну машину,
- визуелизацију процеса.

Систем омогућава директну трансформацију спецификације у извршиву апликацију, чиме се остварује основни циљ рада.

Глава 7

Закључак

У овом раду представљен је језик специфичан за домен BPML (Business Process Modeling Language), намијењен моделовању пословних процеса и аутоматској генерацији full-stack софтверских система. Основна идеја рада заснива се на централизацији процесне логике у декларативном моделу, који се затим трансформише у извршиву backend и frontend апликацију.

У оквиру рада реализовани су сљедећи кључни елементи:

- дефинисање формалне граматике језика употребом TextX алата,
- имплементација семантичке валидације модела,
- развој генератора backend система заснованог на Spring Boot платформи,
- развој генератора frontend система заснованог на React и TypeScript технологијама,
- имплементација процесне машине за управљање животним циклусом процеса,
- визуелизација процесних модела.

Предложени приступ омогућава да се једном дефинисана спецификација користи као централни извор истине за цјелокупан систем. На тај начин се објезбеђује конзистентност између модела и имплементације, смањује количина ручно писаног инфраструктурног кода и убрзава развој нових система.

Анализа система показала је да BPML успјешно подржава:

- моделовање стања и транзиција,
- управљање задацима са fork-join семантиком,
- контролу приступа засновану на улогама,
- повезивање ентитета са процесним инстанцама,
- статичку валидацију модела прије генерације кода.

Систем омогућава брзу изградњу прототипа и продукционих апликација у домену управљања пословним процесима. Додатно, раздвајање доменске спецификације од техничке имплементације омогућава лакше одржавање и проширење система.

Ипак, уочена су и одређена ограничења. Тренутна верзија језика не подржава условне транзиције, динамичко креирање улога, као ни графички едитор за моделовање. Такође, систем не обухвата напредне механизме анализе и оптимизације процеса.

Закључно, BPML представља практичну примјену концепата језика специфичних за домен и моделом вођеног развоја у области пословних процеса. Рад показује да је

могуће изградити систем који омогућава директну трансформацију формалне спецификације у извршиву full-stack апликацију, чиме се остварује значајна уштеда времена и повећање конзистентности система.

7.1 Будући рад

Једно од планираних проширења типског система јесте увођење подршке за датумске и временске типове (нпр. date/datetime). Ово проширење би омогућило природније моделовање доменских података као што су рокови, датуми креирања и периоди важења, као и доследно мапирање на типове циљних платформи (нпр. Java time API и одговарајуће TypeScript репрезентације).

Осим тога, будући рад може обухватити:

- увођење условних израза у транзицијама,
- подршку за сложене типове података и односе између ентитета,
- развој графичког едитора који генерише BPML спецификацију,
- интеграцију са постојећим BPM алатима,
- увођење аналитичких механизама за праћење перформанси процеса,
- подршку за дистрибуирано извршавање процеса.

Списак слика

Слика 1 Слика 1: Логичка архитектура BPML система 20
Слика 2 Слика 2: Структура BPML имплементације 20
Слика 3 Слика 3: Пример процесног модела одобравања фактуре 21
Слика 4 Слика 4: Fork-join семантика задатака 21
Слика 5 Слика 5: Поједностављени UML метамодел BPML језика 21

Списак листинга

Листинг 1 Исјечак TextX граматике за BPML језик 12
Листинг 2 Формални EBNF приказ синтаксе BPML језика 15
Листинг 3 Исјечак семантичке валидације у Python-у 19
Листинг 4 Исјечак генерисаног Java кода за извршавање транзиција 25
Листинг 5 Исјечак генерисаног Java кода за завршетак задатка 26
Листинг 6 Исјечак генерисане React компоненте за листу ентитета 30

Списак табела

Табела 1 Систематизација кључних ријечи BPML језика 17

Списак коришћених скраћеница

Скраћеница	Опис
API	Application Programming Interface (апликациони програмски интерфејс)
AWS	Amazon Web Services (Амазон веб сервиси)
CI/CD	Continuous Integration / Continuous Delivery (континуирана интеграција / континуирана испорука)
CORS	Cross-Origin Resource Sharing (размена ресурса између извора и дестинације различитог порекла)
CSS	Cascading Style Sheets (језик за описивање стилова)
DOM	Document Object Model (објектни модел документа)
DTO	Data Transfer Object (објекат за пренос података)
HTTP	HyperText Transfer Protocol (протокол за пренос хипертекста)
JSON	JavaScript Object Notation (формат за размену података)
JWT	JSON Web Token (сигурносни токен заснован на JSON формату)
RLS	Row-Level Security (сигурност на нивоу реда)
REST	Representational State Transfer (скуп правила за комуникацију између клијента и сервера)
RPC	Remote Procedure Call (позив удаљене процедуре)
SQL	Structured Query Language (структурирани упитни језик)
TLS	Transport Layer Security (безбедност транспортног слоја)
UML	Unified Modeling Language (језик за моделовање дијаграма)
URL	Uniform Resource Locator (јединствени идентификатор и локатор ресурса)
UI	User Interface (кориснички интерфејс)
UUID	Universally Unique Identifier (универзално јединствени идентификатор)
WAL	Write-Ahead Logging (записивање операција унапред)

Списак коришћених појмова

Појам	Објашњење
Асихнрони рад	Рад који се изводи независно од главног тока извршавања, омогућавајући наставак других операција без чекања на његов завршетак.
Bucket (S3)	Логичка јединица за складиштење у AWS S3 сервису, која организује фајлове у облаку.
Read-Only	Режим рада у коме су подаци само за читање, без могућности измене.
Cross-platform	Способност софтвера да се извршава на више различитих оперативних система из истог кода.
Connection pool	Механизам за управљање и поновну употребу веза са базом података како би се побољшале перформансе апликације.

Биографија

Овде написати своју кратку биографију.

Литература

[1] И. Дејановић, *Језици сӣецифични за домен*. Факултет техничких наука, Нови Сад, 2021.

[2] I. Dejanović, R. Vaderna, G. Milosavljević, и Ž. Vuković, „TextX: A Python tool for Domain-Specific Languages implementation“, *Knowledge-Based Systems*, том 115, 2017, doi: 10.1016/j.knosys.2016.10.023.

[3] M. Fowler, *Domain-Specific Languages*. Addison-Wesley, 2010.

[4] M. Mernik, J. Heering, и A. M. Sloane, „When and How to Develop Domain-Specific Languages“, *ACM Computing Surveys*, том 37, изд. 4, стр. 316–344, 2005, doi: 10.1145/1118890.1118892.

[5] D. C. Schmidt, „Model-Driven Engineering“, *IEEE Computer*, том 39, изд. 2, стр. 25–31, 2006, doi: 10.1109/MC.2006.58.

[6] M. Dumas, M. La Rosa, J. Mendling, и H. Reijers, *Fundamentals of Business Process Management*, 2nd изд. Springer, 2018. doi: 10.1007/978-3-662-56509-4.

[7] M. Weske, *Business Process Management: Concepts, Languages, Architectures*, 2nd изд. Springer, 2012. doi: 10.1007/978-3-642-28616-2.

[8] W. M. P. van der Aalst, „The Application of Petri Nets to Workflow Management“, *Journal of Circuits, Systems, and Computers*, том 8, изд. 1, стр. 21–66, 1998, doi: 10.1142/S0218126698000043.

[9] W. M. P. van der Aalst, *Process Mining: Data Science in Action*, 2nd изд. Springer, 2016. doi: 10.1007/978-3-662-49851-4.

[10] J. C. Cleaveland, „Program Generators with XML and Java“, *IEEE Software*, том 18, изд. 5, стр. 25–31, 2001, doi: 10.1109/52.951493.

Грешке у литературним наводима

Овде се налази списак грешака у литературним наводима које морате исправити у `literatura.bib` фајлу.

1. Референци `pyflies` недостаје поље `urldate`